UNIT 3 · TOPIC 3.3

Protecting Networks: Segmentation

Assignment, Answer Key

Teacher answer key, 18 points total.

Practice Questions

3.3.a · VLANs and network segmentation

1. A host on VLAN 10 wants to send data to a host on VLAN 20. What device must the traffic pass through? (1 pt)

- ○ A. A hub
- **● B. A router or layer-3 switch**
- ○ C. Another VLAN-tagged host
- ○ D. The same switch port

Why: VLANs are separate broadcast domains. Traffic crossing VLANs must pass through a router (inter-VLAN routing) where firewall rules are applied.

2. Which of the following are TRUE about VLANs? Select ALL that apply. (3 pts)

- **■ A. VLANs are logical, not physical, separations**
- **■ B. A single switch can host multiple VLANs simultaneously**
- □ C. VLANs prevent all forms of attack with no additional configuration
- **■ D. Broadcast traffic on one VLAN is not visible to hosts on a different VLAN**

Why: VLANs are logical, one switch supports many, and broadcasts are confined per VLAN. VLANs alone do not prevent attacks, firewall rules at the routing boundary are also required.

3. An attacker compromises a guest device on the visitor VLAN. The corporate file server is on the server VLAN. Assuming correct VLAN configuration and restrictive firewall rules, what can the attacker reach from the guest device? (1 pt)

- ○ A. The file server and all other internal servers
- ○ B. Only trunk ports
- ○ C. Any device on any VLAN because they share the same switch
- **● D. Only devices on the same guest VLAN; the file server is unreachable**

Why: Segmentation confines the attacker to their VLAN. The file server is in a different VLAN and the router/firewall blocks cross-VLAN access to internal servers from the guest VLAN.

4. What is the purpose of a trunk port? (1 pt)

- ○ A. To block all VLAN traffic
- **● B. To carry 802.1Q-tagged frames for multiple VLANs on a single physical link**
- ○ C. To connect an end-user workstation to exactly one VLAN
- ○ D. To assign IP addresses to hosts

Why: Trunk ports carry frames tagged with VLAN IDs (802.1Q), allowing multiple VLANs to share an uplink between a switch and a router.

5. Name the attack technique where an intruder who has compromised one host attempts to reach additional hosts deeper in the network. (1 pt)

Acceptable answers: lateral movement; lateral; pivoting; hopping

Why: Lateral movement (also called pivoting) is the technique of moving from one compromised host to another. Segmentation limits it by requiring the attacker to break through a new access control boundary at each VLAN boundary.

3.3.b · Traffic isolation and zero-trust segmentation

6. What is the core paradigm of a Zero Trust Architecture (ZTA)? (1 pt)

- ● **A. Never trust, always verify, regardless of where the connection request originates**
- ○ B. Implicit trust is granted once a device successfully authenticates to the VPN
- ○ C. Trust but verify internal hosts based on their physical network connection
- ○ D. Trust all devices connected to the internal Management VLAN

Why: Zero Trust assumes that all network traffic is untrusted by default, whether it originates from outside the perimeter or inside the local network. It requires explicit, continuous verification.

7. Which of the following are core practices of a Zero Trust Network Architecture? Select ALL that apply. (2 pts)

- ■ **A. Verifying device health and compliance posture before granting resource access**
- □ B. Permitting unlimited lateral movement within the server VLAN
- ■ **C. Using microsegmentation to limit the blast radius of a breach**
- □ D. Granting broad access privileges based on physical location in the office

Why: Zero Trust relies on explicit posture verification and microsegmentation. It removes implicit trust based on network location or physical presence, and blocks unrestricted lateral movement.

8. In a Private VLAN (PVLAN) setup, what type of port is completely blocked from communicating with any other ports in the same PVLAN, except for promiscuous ports? (1 pt)

- ○ A. Community port
- ● **B. Isolated port**
- ○ C. Trunk port
- ○ D. Secondary port

Why: Isolated ports in a Private VLAN cannot communicate with other isolated ports or community ports. They can only send and receive traffic from promiscuous ports (typically routers or gateways).

9. Name the security technique that applies fine-grained security policies to individual hosts or workloads, treating each device as its own isolated boundary to limit lateral movement. (1 pt)

Acceptable answers: microsegmentation; micro-segmentation; micro segmentation

Why: Microsegmentation divides the network into fine-grained zones, down to individual hosts or application containers, to prevent unauthorized east-west (lateral) traffic.

Coding Assignments (Python 3)

3.3.a · VLANs and network segmentation

3 pts · auto-graded on the site

Coding problem, 3.3.a: VLAN segmentation flow filter

Skill: Mitigate Risk · **Maps to:** 3.3.A · **Language:** Python 3 (pytest)

Problem

A network switch enforces VLAN segmentation using the following default-deny policy:

- Traffic between two hosts **on the same VLAN** is **allowed**.

- Traffic from VLAN 20 (Employee) to VLAN 30 (Server) is **allowed** (business policy exception).
- All other cross-VLAN traffic is **blocked**.

Given a mapping of host names to VLAN IDs and a list of traffic flows, return the list of flows that are **blocked** by these rules.

Function contract

Implement `solve(vlan_map: dict, flows: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `vlan_map`: `dict[str, int]`, maps each host name to its VLAN ID (integer).
- `flows`: `list[tuple[str, str]]`, each tuple is `(src_host, dst_host)`.

Output

A `list[tuple[str, str]]` of flows that are blocked, in the same order they appear in the input. Do not include allowed flows.

Examples

```
vlan_map = {"WS-01": 20, "SRV-01": 30, "GUEST-01": 40, "MGMT-01": 10}
flows = [("WS-01","SRV-01"), ("GUEST-01","SRV-01"), ("WS-01","WS-01")]

solve(vlan_map, flows)
-> [("GUEST-01","SRV-01")]
# WS-01->SRV-01: VLAN 20->30, allowed by business exception
# GUEST-01->SRV-01: VLAN 40->30, blocked (no exception)
# WS-01->WS-01: same VLAN, allowed
```

Constraints

- Deterministic, offline, standard library only.
- All host names in `flows` are guaranteed to be in `vlan_map`.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
from typing import List, Tuple, Dict


def solve(vlan_map: Dict[str, int], flows: List[Tuple[str, str]]) -> List[Tuple[str, str]]:
    blocked = []
    for src, dst in flows:
        v_src = vlan_map[src]
        v_dst = vlan_map[dst]
        if v_src == v_dst:
            continue
        if v_src == 20 and v_dst == 30:
            continue
        blocked.append((src, dst))
    return blocked
```

3.3.b · Traffic isolation and zero-trust segmentation

3 pts · auto-graded on the site

Coding problem, 3.3.b: Zero Trust Access Proxy

Skill: Mitigate Risk · **Maps to:** 3.3.B, 3.3.C · **Language:** Python 3 (pytest)

Problem

In a Zero Trust Network Architecture (ZTNA), network access is never implicitly granted based on a user's location. Instead, every request to access a corporate resource must be explicitly authenticated, authorized, and checked for health compliance.

You are tasked with implementing the evaluation logic for a Zero Trust access proxy. The proxy evaluates a batch of access requests against a central policy and filters out any requests that fail to meet authorization, location, or posture requirements.

Function contract

Implement `solve(requests: list, policy: dict) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `requests`: A list of dictionaries representing connection requests. ``python requests = [{ "id": "REQ-001", "user": "alice", "role": "doctor", "resource": "ehr_db", "ip": "192.168.1.50", "device_posture": {"antivirus_ok": True, "disk_encrypted": True} }]
- `policy`: A dictionary detailing the security rules. ``python policy = { "allowed_roles": { "ehr_db": ["doctor", "admin"], "mri_machine": ["admin", "technician"] }, "requires_internal": ["ehr_db", "mri_machine"], "requires_posture": ["ehr_db"], "internal_prefixes": ["192.168.1.", "10.0."] }

Output

A list of strings representing the `id`s of requests that are **allowed**, maintaining their original order.

Examples

```
requests = [
    {
        "id": "REQ-001",
        "user": "alice",
        "role": "doctor",
        "resource": "ehr_db",
        "ip": "192.168.1.50",
        "device_posture": {"antivirus_ok": True, "disk_encrypted": True}
    },
    {
        "id": "REQ-002",
        "user": "bob",
        "role": "guest",
        "resource": "ehr_db",
        "ip": "192.168.1.60",
        "device_posture": {"antivirus_ok": True}
    }
]
# REQ-001 is allowed (role is doctor, IP matches internal prefix, posture checks are all True).
# REQ-002 is denied (guest role is not allowed on ehr_db).

solve(requests, policy) -> ["REQ-001"]
```

Constraints

- Deterministic, offline, standard library only.
- All requests have the required keys.
- Must pass all cases in `tests.py`.

Reference solution:

```
from typing import List, Dict, Any


def solve(requests: List[Dict[str, Any]], policy: Dict[str, Any]) -> List[str]:
    allowed_ids = []
    allowed_roles = policy.get("allowed_roles", {})
    requires_internal = policy.get("requires_internal", [])
    requires_posture = policy.get("requires_posture", [])
    internal_prefixes = policy.get("internal_prefixes", [])

    for req in requests:
        rid = req["id"]
        res = req["resource"]
        role = req["role"]
        ip = req["ip"]
        posture = req.get("device_posture", {})

        if res not in allowed_roles:
            continue
        if role not in allowed_roles[res]:
            continue

        if res in requires_internal:
            ip_ok = False
            for prefix in internal_prefixes:
                if ip.startswith(prefix):
                    ip_ok = True
                    break
            if not ip_ok:
                continue

        if res in requires_posture:
            posture_ok = True
            for val in posture.values():
                if not val:
                    posture_ok = False
                    break
            if not posture_ok:
                continue

        allowed_ids.append(rid)

    return allowed_ids
```